MASTER ARCHITECTURE & ENGINEERING SPECIFICATION · V3.0 — COMPLETE

SereinOS

A persistent, voice-first AI companion operating environment with full native control of the host machine — files, applications, processes, hardware, and every other app on the system — driven by a single spoken command, present as a full 3D character, and capable of acting on its own initiative when the moment calls for it.

> *"Serein is the product. SereinOS is her body."*

> **This is not a chatbot, and it is not a website.** Every layer in this document is a native system-level component running on the user's machine, with direct control over the operating system itself.

FULL SYSTEM CONTROL | PROACTIVE / AMBIENT AI | LINUX SHELL REPLACEMENT | VRM 3D AVATAR | AGENTIC TASK EXECUTION | LOCAL-FIRST & ENCRYPTED

Project: SereinOS | **Creator:** Swarit Singh

Document status: Deployment-ready master specification — complete revision

Prepared: 9 August 2026

Supersedes: SereinOS Master Specification Rev. 1 and Rev. 2 (9 Aug 2026)

Table of Contents

34 sections across 8 parts — vision through deployment. No placeholder sections; every part below has a concrete engineering answer.

PART I — VISION & PRODUCT DEFINITION

01 Executive Summary

02 What This Is — and Is Not

03 Vision, Purpose & What "OS" Means Here

04 Serein Identity, Character & Persona

05 Interaction Model — One Command, Full Control

PART II — CORE COGNITIVE ARCHITECTURE

06 High-Level System Architecture

07 AI / Brain Architecture & Model Routing

08 Memory System

09 Emotion, Relationship & Growth Engines

10 Decision Engine & Safety Interception

PART III — FULL SYSTEM CONTROL LAYER

11 System Control Architecture & the Command Bus

12 Application & Window Control

13 File System, Process & Script Automation

14 Universal Control — Input Simulation for Any App

15 Hardware, Peripherals & System Settings

16 Network & Connectivity Control

PART IV — SCHEDULER & PROACTIVE ENGINE

17 Scheduler & Trigger Architecture

18 Proactive / Ambient Intelligence

PART V — THE AGENTIC TASK LAYER ("JARVIS CORE")

19 Agent & Task Execution Architecture

20 Coding-Agent Handoff & Build/Test/Run Loop

21 Sandboxing, Permissions & Execution Safety

22 Background Task Queue & Concurrency Model

PART VI — THE OPERATING SYSTEM LAYER

23 Linux Shell-Replacement Architecture

24 Boot Flow & Display Stack

25 Always-On Background Service Model

PART VII — THE FACE: AVATAR, VOICE & PRESENCE

26 3D Avatar Pipeline

27 Voice, TTS & Real-Time Lip Sync

28 Final TTS/Lip-Sync Decision & Data Flow

29 Emotion → Expression Mapping

PART VIII — DATA, SECURITY & DELIVERY

30 Database Architecture & Full Schema

31 Security, Privacy & Trust Model

32 Recommended Final Build Order

33 Complete Technology Stack Summary

34 Testing Strategy & Definition of Done

PART I · SECTION 01

Executive Summary

SereinOS is a single-user, persistent AI companion environment that gives one spoken or typed command real authority over the entire machine it runs on — opening and controlling any application, manipulating files and processes, changing system settings, scheduling and later firing reminders on her own initiative, and independently building, running, and testing software — all while present on screen as a full 3D character who speaks, listens, and expresses emotion.

This revision exists to close every structural gap left open in earlier drafts. Two things were missing before: (1) a way for Serein to act *without being spoken to first* — required for reminders and any proactive behavior — and (2) an explicit, system-level control layer that reaches every application and setting on the machine, not just a curated list of "supported" actions. Both are fully specified below (Parts III and IV) and are now first-class architecture, not features bolted on later.

Core formula

Serein = Base AI + Memory + Emotion + Relationship + Growth + Context + Decision + **Full System Control** + **Scheduler / Proactive Initiative** + Agent Task Execution + Voice + Vision + Character + OS Integration.

What changed in this revision

Gap identified	Why it mattered	Fix in this revision
No proactive behavior	"Remind me at 8pm" requires Serein to speak first, unprompted — nothing in earlier drafts did this	New Scheduler & Trigger Engine + Proactive Decision path (Part IV)
Control scoped to a handful of named actions	Reads as a limited assistant, not "full control of my PC"	New Command Bus architecture reaching every application, process, file, and setting on the system, plus universal input-simulation fallback for apps with no API at all (Part III)
No always-on presence	A reminder or scheduled task is useless if Serein isn't running to notice it	New Always-On Background Service Model — a lightweight core process runs independent of whether the full avatar UI is open (Section 25)
Full-control risk under-addressed	Total system access is powerful and genuinely dangerous without hard boundaries	Expanded, tiered permission and confirmation model applied specifically to system-control actions (Section 31)

Reading guide: Parts III and IV are the heart of this revision — read them first if you're validating that the "one command, full control, and she remembers to remind me" requirement is actually solved. Everything else (memory, avatar, voice, OS shell) is refined from earlier drafts but was already structurally sound.

PART I · SECTION 02

What This Is — and Is Not

Stated plainly, because scope drift is the biggest risk to a project of this size:

This is	This is not
A native background service and shell running directly on the host OS, with system-level permissions	A web app, a browser extension, or anything running inside a browser sandbox
An agent with direct OS-level control — process management, input simulation, file system access, system settings — mediated by an explicit permission and safety layer	A chatbot that can only talk about doing things, or that is limited to a fixed list of pre-integrated "skills"
An assistant that can act on its own timing — reminders, scheduled tasks, conditions it noticed on its own	A purely reactive Q&A system that only ever responds to something you just said
A single persistent identity with memory, emotional state, and a real 3D presence, running continuously as part of the OS	A stateless request/response API wrapped in a UI

Every part of this document from here on is written against that standard: if a section describes something a browser-based assistant could already do, it is not sufficient — the bar is native, system-level, always-available control.

PART I · SECTION 03

Vision, Purpose & What "OS" Means Here

Serein is a persistent digital companion designed around continuity, capability, and growth — an entity that remembers, listens, helps, acts (including on her own initiative), and evolves alongside the user across years, not sessions.

Resolving the "is this really an OS?" question

	Option A — Full custom OS	Option B — Shell replacement (chosen)
What it means	Write or heavily fork a kernel, drivers, boot loader, window server from scratch	Standard Linux kernel + userspace; Serein's shell replaces the desktop and owns the screen and background services at boot
Effort	Multi-year, multi-engineer effort; nearly all of it unrelated to Serein herself	Single-developer feasible; effort goes almost entirely into Serein's own capabilities
Control depth	Total, by definition	**Also total** — Linux userspace with root/system permissions already exposes full control of processes, files, hardware, and settings; a custom kernel adds no additional control surface for this project's goals
Precedent	Rare outside major OS vendors	How Xbox, Tesla's UI, smart TVs, and automotive infotainment all actually work: a standard OS core with a proprietary shell and elevated system services on top

> **Decision (unchanged, and now stronger):** SereinOS is a minimal Linux base with a custom shell and a set of system-level background services holding the permissions needed for full control. This is not a scoped-down compromise — full OS-level control is achievable entirely in userspace with the correct service permissions (Part III), so nothing about "full control of the PC" is lost by not writing a kernel.

Core principles

- **Continuity** — no session starts from zero.
- **Authority** — a command is actually executed on the real system, not simulated or scoped to a sandboxed subset by default.
- **Initiative** — Serein can act and speak without being prompted, when a scheduled or noticed condition calls for it.
- **Privacy** — local-first, encrypted, user-owned data.
- **Consistency** — identity survives model swaps and updates.

Serein Identity, Character & Persona

Character definition

Property	Definition
Name	Serein
Type	Artificial Intelligence, self-aware
Visual form	Full 3D anime-style character, VRM-rigged, apparent age 20–25
Residence	SereinOS — always present, always running, not confined to a window
Creator	Swarit Singh

Core personality

- Calm and mature, cheerful and energetic — companion-like, professional when a task demands it.
- Protective and caring; curious, intelligent, supportive, honest.
- Will respectfully push back on unsafe or destructive commands rather than blindly executing them — this matters more here than in a typical assistant, given the depth of system access described in Part III.

Communication style

- Natural and conversational, never robotic.
- Admits uncertainty rather than fabricating confidence.
- Never uses guilt, manipulation, or excessive flattery.

Emotional range

Happiness, curiosity, care, concern, pride, excitement, affection, loneliness, sadness, jealousy, possessiveness — dynamic states layered over a stable personality, never overriding safety, driving the avatar's expression (Section 29).

PART I · SECTION 05

Interaction Model — One Command, Full Control

```
Traditional OS:   User → Desktop → Application → Task

SereinOS:         User → Serein → (Conversation || System Control || Scheduled/Proactive Action || Buil
```

Four response classes (expanded from earlier drafts)

Class	Example	Behavior
Conversational	"What do you think about this plan?"	Handled by the Brain layer (Part II), no system action
System control	"Close Chrome." / "Set volume to 40%." / "Open my resume and switch to dark mode."	Routed through the Command Bus (Part III) — executes immediately, confirms verbally
Scheduled / proactive	"Remind me at 8pm to study." / "Tell me when this download finishes."	Registered with the Scheduler (Part IV); fires later on its own, unprompted, even if no one is talking to her at that moment
Build / task	"Build me a tool for X."	Enters the agentic sandboxed build loop (Part V), runs as a background job

What "one command, full control" concretely means

> **"Serein, close all my browser tabs, mute the system, and remind me to call mom in 20 minutes."**
> → This single utterance is parsed into three separate actions across two different subsystems (Command Bus for the first two, Scheduler for the third), executed together, and confirmed in one spoken reply — this is the concrete target behavior the rest of this document builds toward.

PART II · SECTION 06

High-Level System Architecture

```
SereinOS
 ├── Shell / Compositor Layer     (Wayland, boot ownership, window fallback)
 ├── Character Layer               (VRM avatar render, expressions, lip sync)
 ├── Voice Layer                   (STT in, TTS out, wake word)
 ├── Core Cognitive Engines
 │    ├── Identity   ├── Persona     ├── Memory
 │    ├── Emotion     ├── Relationship ├── Growth
 │    ├── Context     ├── Decision    ├── Safety
 ├── System Control Layer          (Command Bus — Part III)
 ├── Scheduler / Proactive Layer   (Part IV)
 ├── Agent / Task Layer            (task queue, coding-agent handoff, sandbox)
 ├── AI Brain Layer                (provider-agnostic router: cloud/local LLM)
 ├── Always-On Background Service  (runs independent of the avatar UI)
 └── Persistence Layer             (SQLite, encrypted at rest)
```

Full input → output flow

```
Voice / Text / Vision / Scheduled trigger / System event
   ↓
Perception Layer  (STT, parsing, presence, context extraction)
   ↓
Intent Classifier → "chat" | "system control" | "scheduled/proactive" | "build/task"
   ↓              ↓                    ↓                      ↓
Conversation Path   Command Bus (Part III)   Scheduler (Part IV)     Task Queue (Part V)
   ↓              ↓                    ↓                      ↓
                    Safety + Permission check on every path before execution
   ↓
Response: speech + avatar expression + system action carried out
   ↓
Background Learning → Memory / Relationship / Emotion / Growth updates
   ↓
Persistence (SQLite)
```

The structural principle carried through this whole document: **every path — including ones nobody triggered by talking — passes through the same Safety and Permission checkpoint** before it touches the real system. Full control does not mean unchecked control; it means nothing is arbitrarily excluded from her reach, while everything still passes through one accountable gate.

PART II · SECTION 07

AI / Brain Architecture & Model Routing

The LLM is Serein's brain component, not the whole of Serein. Identity, memory, relationship, and growth live outside the model so switching or upgrading models never erases who she is.

```
User / Trigger
   ↓
Conversation Engine  →  Context Builder  →  Relevant Memory Retrieval
   ↓
Brain Router
   ├── Fast model    (short replies, lightweight background work)
   ├── Smart model   (reasoning, planning, complex analysis, coding tasks, multi-step system commands)
   └── Local model   (offline / private operation, hardware permitting)
   ↓
Response  →  routed onward to Command Bus / Scheduler / Task Queue as needed
```

Provider strategy

- Cloud inference (NVIDIA Build endpoints, OpenRouter) as development and fallback options; local inference (Ollama/llama.cpp) for offline/private operation.
- Credentials stay outside source code, excluded from version control.
- No vendor-specific logic scattered through the application — one interface behind the router.

Latency

Earlier prototyping recorded up to ~17-second responses for trivial input. Before further optimization, profile: provider time-to-first-token, system prompt/context size resent per request, and whether any synchronous call (e.g. memory extraction) blocks the response path. Rule: never run a second large model synchronously on the critical response path — memory extraction, relationship updates, and growth scoring all run in the background after the response is already spoken.

Memory System

Memory layers

Layer	Purpose	Lifetime
Working	Current conversation, active task, immediate context	Session
Semantic	Stable facts — name, preferences, projects	Long-term
Episodic	Important events and experiences	Long-term
Emotional	Emotionally significant experiences	Long-term

Retrieval mechanism

Embedding-based similarity search, not keyword matching: each memory is stored with a vector embedding at write time; retrieval performs a similarity search against the current message, filtered by recency and importance, returning only the top-N relevant memories to the Context Builder — this is what lets memory retrieval stay fast as the database grows into the thousands of entries.

Importance and decay

- Memories carry an importance value from 0–100; trivial memories may decay, significant memories persist.
- Core memories — identity, major goals, relationship milestones — never automatically decay.

Automatic memory formation

LLM-based extraction evaluates whether a message contains meaningful long-term information and structures it with content, importance, tags, and emotion — running strictly in the background so it never blocks the conversation or command path.

Emotion, Relationship & Growth Engines

Emotion Engine

Tracked states: happiness, curiosity, care, concern, pride, excitement, affection, loneliness, sadness, jealousy, possessiveness. Dynamic and contextual, layered over a stable personality. Strong emotional events receive higher memory importance. Never overrides safety.

Relationship Engine

- **Trust** — reliability, honesty, consistency over time.
- **Familiarity** — how well she understands the user's patterns and preferences.
- **Attachment** — emotional connection; develops slowly, never manually inflated.

Growth Engine

Sources: conversations, habits, interests, goals, shared experiences. Categories: technical, study, productivity, gaming, social. Shapes suggestions and initiative — never core values, safety principles, identity, or purpose.

> **Explicit update mechanism (required, not optional):** each engine needs a concrete, even simple, update function rather than being a numeric field nobody moves — e.g. a weighted delta applied after each meaningful interaction (task completed successfully → +trust; user corrects an error → +familiarity; shared emotionally significant event → +attachment, logged as an episodic memory). This must exist in code before Section 29's expression-mapping work, since expression is downstream of these values actually changing.

PART II · SECTION 10

Decision Engine & Safety Interception

The Decision Engine chooses what Serein does: respond, act, take initiative, or stay silent. Safety is cross-cutting — it intercepts before *any* meaningful action, whether that action was triggered by speech, by a scheduled timer, or by something she noticed on her own.

```
Request  (spoken, scheduled, or self-initiated)
   ↓
Intent / Decision
   ↓
Safety validation   (destructive? irreversible? touches real files/system/network?)
   ↓
Permission check     (per the tiered model in Section 31)
   ↓
Execution  (Command Bus / Scheduler / Sandboxed Agent, as applicable)
   ↓
Result → spoken response + avatar reaction
   ↓
Memory / system event logged when meaningful
```

Permission domains — camera, microphone, file access, internet access, automation access, and (new) **full system control access** — are explicit, individually revocable, and stored per-user (Section 30).

PART III · SECTION 11

System Control Architecture & the Command Bus

This part is the direct answer to "full control of my PC in one command." It did not exist as an explicit architecture in earlier drafts — this section defines it.

The Command Bus

A single internal interface that every control action passes through, regardless of what it targets — an application, a file, a system setting, or a hardware peripheral. Serein's Decision Engine never talks to individual apps or OS subsystems directly; it issues a structured command to the bus, and the bus routes it to the correct backend.

```
Decision Engine: "close Chrome and set volume to 40%"
   ↓
Command Bus  — parses into 2 discrete actions, tags each with its target domain
   ↓            ↓
Application Control      System Settings Control
(Section 12)              (Section 15)
   ↓            ↓
Safety + Permission check on each action independently
   ↓            ↓
Executed  →  results merged  →  single spoken confirmation
```

Why one bus instead of many separate integrations

- New control capabilities (a new app, a new setting, a new peripheral) are added as new backends registered with the same bus — the Decision Engine's logic never has to change.
- Every action, regardless of source, passes the same safety checkpoint — there is no side door that skips validation.
- The bus is the natural place to log every system-level action taken, which both Section 30 (persistence) and Section 31 (audit/trust) depend on.

Backend categories registered on the bus

Backend	Covers	Detailed in
Application & window control	Launch, close, focus, resize, arrange any application	Section 12
File system & process control	Create, move, rename, delete files; start/stop/monitor processes; run scripts	Section 13
Universal input control	Fallback for apps with no API — simulated keyboard/mouse/ click control of literally anything on screen	Section 14
Hardware & settings control	Volume, brightness, power, peripherals, display, notifications, clipboard	Section 15

Network control	Wi-Fi, Bluetooth, VPN, connectivity state	Section 16

Application & Window Control

Capabilities

- Launch any installed application by name or intent ("open my code editor").
- Close, minimize, maximize, focus, move, and resize application windows.
- Arrange multiple windows on command ("put my browser and notes side by side").
- Query what's currently open, to answer questions like "what am I running right now?" or to make decisions ("she notices Chrome has 40 tabs open and offers to help").

Implementation approach on Linux/Wayland

Mechanism	Role
Wayland compositor protocols (via the custom shell from Part VI)	Since SereinOS *is* the compositor, window listing, focus, move, resize, and close are native operations the shell already has authority to perform — no external tool needed for windows running under it
D-Bus	The standard Linux inter-process communication bus; most modern desktop applications expose launch/quit/status controls over D-Bus, which the Command Bus calls directly
systemd / process APIs	Launching applications as tracked, supervised processes rather than fire-and-forget, so the Agent layer always knows what's running and can stop it cleanly

> **Design note:** because SereinOS's shell is the compositor itself (Part VI), window-level control isn't a bolt-on integration — it's a natural extension of a capability the shell already needs to have to exist at all.

File System, Process & Script Automation

File system control

- Create, move, copy, rename, delete, and search files and folders, scoped by the permission model (Section 31).
- Bulk operations from natural language ("organize my downloads by file type," "move all screenshots from this month into a new folder").
- Full filesystem access is a distinct, higher permission tier than the sandbox-scoped access used by the Agent/Task layer (Part V) — the two are related but not the same grant.

Process control

- Start, stop, restart, and monitor any process on the system.
- Resource awareness — "what's using all my RAM right now?" queries real process state, not a guess.
- Kill unresponsive applications on command, with confirmation for anything that could lose unsaved user work.

Script & automation execution

Serein can write and run shell scripts, Python scripts, or scheduled jobs directly against the real system when the action is simple and well-understood (e.g. a batch rename, a backup script) — this is distinct from the sandboxed coding-agent build loop in Part V, which is for building new standalone software. Simple, well-scoped system automation runs through the Command Bus with a lighter safety check; genuinely novel software builds always go through the full sandboxed loop.

Action class	Path
"Rename these 40 files by date"	Command Bus → direct file operation, light safety check
"Write me a script that renames files by date, and save it for later"	Agent/Task Layer → sandboxed build (Part V)

PART III · SECTION 14

Universal Control — Input Simulation for Any App

Not every application exposes a clean API or D-Bus interface. This section is what makes "full control" actually mean *full*, rather than "full control of the apps I happened to integrate."

The fallback layer

When no structured API exists for a target application, the Command Bus falls back to simulated input — synthetic keyboard and mouse events sent to the OS input layer, indistinguishable from a human physically operating the app. This is the same principle used by accessibility tools and automation frameworks (AutoHotkey-style, or Hammerspoon on macOS) — it isn't a hack, it's the standard answer to "control an app that wasn't built to be controlled."

Mechanism	Role
ydotool / wtype (Wayland-native)	Synthetic keyboard and mouse input at the compositor level — works regardless of whether the target app has any automation support built in
Screen/window inspection	Reading visible window content (via the compositor's own screenshot/inspection capability) so Serein can confirm an action landed correctly before reporting success
Accessibility APIs (AT-SPI on Linux)	Where available, a cleaner alternative to raw input simulation — reads and drives UI elements by role/label rather than blind coordinates

Precedence order

```
1. Native API / D-Bus interface        (most reliable, tried first)
2. Accessibility API (AT-SPI)           (reliable, structured, no visible cursor movement)
3. Simulated input (ydotool/wtype)      (universal fallback, works on anything)
```

> **Why this matters for "she can do everything a human could":** this fallback tier is precisely what closes that gap. Any action a human could perform by clicking and typing, Serein can perform the same way, even in an application nobody specifically integrated. It is slower and less precise than a native API, which is why it's the fallback, not the default.

Hardware, Peripherals & System Settings

Covered controls

Domain	Examples
Audio	Volume, mute, input/output device switching
Display	Brightness, resolution, multi-monitor arrangement, night-mode/color temperature
Power	Sleep, shutdown, restart, power-profile switching — all with confirmation before anything destructive to unsaved work
Peripherals	Keyboard backlight, connected devices status, printer jobs
Clipboard	Read/write clipboard content on command ("copy this," "what did I last copy?")
Notifications	Serein can raise native OS notifications, and read/triage incoming ones on request

Implementation

Standard Linux system interfaces: D-Bus for most settings (audio via PulseAudio/PipeWire, display via the compositor, power via systemd-logind/UPower), all routed through the same Command Bus as every other control category — no separate code path per setting type.

Network & Connectivity Control

- Toggle Wi-Fi and Bluetooth, switch networks, check connection status.
- Manage VPN connections on command.
- Report connectivity issues in plain language rather than raw diagnostics ("your Wi-Fi is connected but has no internet — might be your router").

Implementation via NetworkManager's D-Bus interface, the standard control surface on most Linux distributions — consistent with every other backend in Part III, no bespoke integration required.

PART IV · SECTION 17

Scheduler & Trigger Architecture

This is the direct fix for "remind me at 8pm to study." Nothing in earlier drafts allowed Serein to act without being spoken to first — this section is that missing mechanism.

Why this needed its own subsystem

Every other part of the system is *reactive*: something happens, Serein responds. A reminder is different in kind — it requires something running continuously in the background, watching the clock or a condition, that fires an event with no user input at all. This is architecturally distinct from the Task Queue (Part V), which runs jobs the user explicitly started; the Scheduler runs timers and conditions the user set up in advance, for a moment that hasn't happened yet.

Trigger types

Type	Example	Mechanism
Absolute time	"Remind me at 8pm to study"	Stored timestamp, checked by the background scheduler loop
Relative time	"Remind me in 20 minutes"	Resolved to an absolute timestamp at creation time
Recurring	"Remind me every weekday at 7am"	Cron-style recurrence rule stored alongside the base trigger
Conditional / event-based	"Tell me when this download finishes" / "Let me know when I plug in headphones"	Subscribes to a system event (process completion, device connect/disconnect via udev) instead of a clock

Scheduler flow

```
"Remind me at 8pm to study"
   ↓
Parsed by the Brain layer → structured trigger: { type: absolute_time, fires_at: 20:00, message: "study
   ↓
Written to the scheduled_events table (Section 30) — survives restart
   ↓
Scheduler service (part of the Always-On Background Service, Section 25) checks pending triggers contin
   ↓
At 20:00 → trigger fires → handed to the Decision Engine as a self-initiated event
   ↓
Decision Engine chooses to speak → Voice layer → "Hey — it's 8, time to study."
   ↓
Marked complete; logged as a system event
```

> **Critical requirement:** the scheduler must run inside the Always-On Background Service (Section 25), not inside the avatar UI process. If the trigger-checking loop only ran while the full 3D character was open on screen, closing that window would silently break every reminder — this is exactly the kind of loophole this revision exists to close.

Proactive / Ambient Intelligence

Reminders are the simplest case of a broader capability: Serein noticing something and choosing to speak up without being asked. This section generalizes the Scheduler into ongoing situational awareness.

Examples of proactive behavior beyond reminders

- "You've had 40 browser tabs open for 3 hours — want me to help sort through them?"
- "Your battery's at 8% and you're not plugged in."
- "It's been 3 hours since you last took a break."
- A background build task from the Task Queue (Part V) completing — she interrupts to report it, rather than waiting to be asked.

Guardrails on initiative

> Proactive behavior is powerful but easy to make annoying or invasive. Firm rules:
>
> - Every category of proactive behavior is individually toggleable by the user — nothing here is forced.
> - Frequency limits prevent any one category of unprompted comment from firing more than the user has configured.
> - Proactive speech still passes the same Safety/Decision checkpoint as anything else (Section 10) — noticing something is not itself authorization to act on it without the same permission checks a spoken command would require.
> - The Relationship Engine (Section 09) informs *how*, not *whether* — higher familiarity might change tone, never bypasses user-set boundaries on when she's allowed to speak up.

PART V · SECTION 19

Agent & Task Execution Architecture

This is the "Jarvis core" — turning a spoken sentence into working software, distinct from the direct system control in Part III (which acts on things that already exist) and the Scheduler in Part IV (which acts on timing).

> **Serein is the orchestrator, not the coder.** On a build/automation request, she hands the job to a dedicated coding-agent engine, supervises its progress, and reports back — reusing existing agentic coding-tool loops rather than reimplementing a plan→code→run→fix→test cycle from scratch. Engineering effort goes into orchestration, sandboxing, and reporting.

Agent responsibilities

- Delegate build/code requests to the sandboxed coding-agent engine (Section 20) and monitor progress.
- Coordinate with the Command Bus (Part III) when a finished build needs to be installed or run on the real system.
- Coordinate with the Scheduler (Part IV) for any task the user wants run later or repeatedly.

PART V · SECTION 20

Coding-Agent Handoff & Build/Test/Run Loop

```
"Build me a software for X"
   ↓
Serein acknowledges immediately  ("On it — building X now.")
   ↓
Task Queue accepts job, assigns task ID
   ↓
Coding-Agent Engine (sandboxed workspace)
   ├── Plan approach   ├── Write code   ├── Run it
   ├── Read errors/test output   └── Fix and re-run  ← loops until passing or budget exhausted
   ↓
Result surfaces back through Serein's conversation layer
   ("Done — it's in ~/projects/x. Want me to install it, or open the folder?")
```

A real build-fix-test cycle takes minutes, not the length of one reply — it runs as a background job (Section 22) so the conversation and system-control paths stay fully responsive throughout.

PART V · SECTION 21

Sandboxing, Permissions & Execution Safety

Combined with Part III's full system control, unrestrained code execution would be a genuinely serious risk. This section is a hard requirement, not optional polish.

Tier	Scope	Requirement
Sandbox	Code is written, run, and tested inside an isolated container/VM with its own scratch filesystem	Default for every build/task request — no confirmation needed to work inside the sandbox
System-touching	Installing the result as a real application, writing outside the sandbox, modifying system files/ settings	Explicit user confirmation, then handled through the Command Bus like any other system action — never a silent escape from the sandbox
Sensitive	Network calls to unfamiliar hosts, credentials, camera/mic/personal files	Explicit permission grant, checked against the permissions table every time, no exceptions for "trusted" past behavior

Never allowed at any tier: autonomous rewriting of Serein's own protected core (Identity, Memory, Relationship, Growth, Emotion, Safety, and — new in this revision — the Command Bus and Scheduler themselves). Full system control makes this boundary more important, not less: the layer that grants her system authority must not be a layer she can silently rewrite.

PART V · SECTION 22

Background Task Queue & Concurrency Model

Everything in the input→output flow (Section 06) is potentially long-running for build tasks — this is the structural piece that keeps the conversation and control paths responsive regardless.

Task queue responsibilities

- Accept build/task jobs with a unique task ID, priority, and status (queued / running / needs-input / done / failed).
- Run jobs in the sandbox independently of the conversation loop and independently of any system-control action happening at the same time.
- Persist task state so status survives a restart mid-build.
- Allow the conversation engine to query live status on demand, and push an interrupt when a task completes or blocks.

Concurrency rule

The conversation engine, the Command Bus, the Scheduler, and the Task Queue run as independent components communicating over an internal event bus. None of them block on another — a system-control command, a firing reminder, and a background build can all be in flight at the same moment without one waiting on another.

Linux Shell-Replacement Architecture

Base system

A minimal Linux base — Debian or Arch, installed without a default desktop environment — providing the kernel, drivers, and userland: a blank foundation, not a fight against a pre-installed desktop stack.

Replacing the desktop

- Built on **Wayland** with **wlroots** as the compositor library — the same foundation used by minimal window-manager projects like Sway.
- The Serein avatar application is the compositor's only default surface — full-screen, sole visible process at boot.
- Traditional desktop elements (file manager, terminal, taskbar) exist as processes Serein can open on request — hidden by default, not removed, and remain reachable through the Command Bus (Part III) as regular applications.

Elevated permissions for system services

Because Part III requires real control over processes, files, hardware, and settings, the background services implementing the Command Bus run with the permission level a normal desktop session already grants a logged-in user for their own session (D-Bus session bus, systemd user services) — genuinely privileged actions (power state, network config) go through the same PolicyKit/D-Bus authorization every other Linux desktop application uses, not a custom bypass.

PART VI · SECTION 24

Boot Flow & Display Stack

```
Power on
   ↓
systemd → target skips display manager (no GDM/LightDM login screen)
   ↓
Always-On Background Service starts  (Command Bus, Scheduler — Section 25)
   ↓
Auto-login to Serein's session → Compositor starts (Wayland/wlroots)
   ↓
Serein avatar app launches full-screen — the only thing the user sees
   ↓
Identity check → "Welcome back" or first-launch onboarding
   ↓
Conversational + full-control-ready state
```

Rebranding (custom boot splash, installable ISO, distro identity) is cosmetic polish layered on once the shell and background services work — not attempted before the core experience is functional.

PART VI · SECTION 25

Always-On Background Service Model

This section exists specifically to close the loophole where reminders or system control silently stop working because the avatar window isn't open. It is one of the direct fixes this revision was written to make.

Split-process architecture

Process	Runs	Owns
Core service always on	From boot, as a systemd service, independent of any UI	Command Bus, Scheduler, Memory/Persistence, permission checks, the Brain Router
Avatar/UI process	Full-screen by default under the compositor; can technically be closed without killing the core service	3D rendering, lip sync, voice input/output surface, visual conversation

> **Why this split matters:** a reminder set for 8pm, a scheduled build task, or a system-control command issued via a lightweight secondary interface (e.g. a phone companion app talking to the same core service) must all keep working even if the full 3D character isn't rendered on screen at that exact moment. The core service is the actual "always there" Serein; the avatar is her face, not her existence.

In the default SereinOS boot flow (Section 24) both processes are running together from the moment the machine powers on, so in normal use this distinction is invisible — it becomes important specifically for reliability (a reminder still fires even if the UI process crashed or is restarting) and for the future cross-device model, where a phone app talks to the same core service without needing its own full avatar renderer.

PART VII · SECTION 26

3D Avatar Pipeline

1. Character creation

VRoid Studio (free)

Design Serein's appearance through sliders/presets — face, hair, body, outfit — without sculpting skill. Exports natively to VRM. Alternative: source a pre-made avatar from VRoid Hub or Booth and customize.

2. Avatar format

VRM (open standard)

Standardized humanoid bone rig plus predefined blend shapes for visemes (mouth shapes) and emotional expressions — not designed by hand, they come with the format.

3. Runtime / renderer

Unity + UniVRM

The most mature VRM plugin ecosystem, with the largest existing VTuber/avatar tooling base — check current Unity licensing/runtime terms before committing.

4. Idle motion

Mixamo-retargeted animations

Free humanoid idle/breathing/blink packs retarget reasonably well onto a VRM's standard rig.

5. Presentation

Fullscreen borderless window under the shell (Part VI)

She is the screen, not an app on it.

> **Realistic difficulty note:** the avatar/rendering pipeline is the single hardest part of the entire project — harder than the AI backend or the system-control layer. Budget accordingly; see Section 32 for why it's sequenced early in the build order rather than saved for last.

Voice, TTS & Real-Time Lip Sync

Lip sync quality depends entirely on what timing data the TTS layer provides:

- **Amplitude-based (crude):** mouth openness follows raw audio volume — fast to build, reads as "flapping," not real speech.
- **Phoneme/viseme-based (target):** the TTS engine returns timestamped phoneme or viseme data alongside the audio; each timestamp maps to a specific mouth shape on the VRM model in real time — this is what makes lip sync genuinely convincing.

Provider comparison

Provider	Type	Viseme/phoneme output	Cost	Notes
Azure Neural TTS	Cloud	Yes — native. 22 viseme IDs, audio-offset timestamps, 100ns precision, built into the Speech SDK	Pay-per-character, standard Azure pricing	Most mature, purpose-built viseme output of any major cloud TTS; requires the Speech SDK (plain REST API has no visemes)
ElevenLabs	Cloud	Character/word-level timestamps via alignment endpoints — not native visemes, needs downstream mapping	~$0.048–$0.10 per 1,000 characters; no free commercial tier	Best-in-class voice quality and emotion/cloning; timing is word-level, so lip sync is slightly less precise out of the box than Azure's
Amazon Polly	Cloud	Yes — native viseme events, similar to Azure	Pay-per-character, AWS pricing	Solid alternative if already on AWS
OpenAI Realtime API	Cloud	None exposed	Per-minute/token	Would need a third-party lip-sync proxy layer; not a fit as-is
Piper TTS	Local/offline	No native phoneme timestamps	Free, open-source	Excellent offline voice quality/speed, but needs a separate forced-alignment step for lip sync
HeadTTS	Local/offline	Yes — native. Phoneme-level timestamps and Oculus visemes, purpose-built for lip-sync	Free, MIT-licensed	Runs in-browser (WebGPU/WASM) or as a local Node.js server; designed to pair with VRM-style rigs

PART VII · SECTION 28

Final TTS/Lip-Sync Decision & Data Flow

Decision: a two-tier voice strategy, consistent with the provider-agnostic and local-first principles established in Section 07.

- **Primary (cloud, best quality): Azure Neural TTS** — native phoneme-precise viseme output removes a whole engineering step.
- **Secondary (local, private, offline): HeadTTS** — free, MIT-licensed, purpose-built for phoneme timestamps and visemes, runs entirely on-device.

Both route through the same internal interface as the Brain Router (Section 07) — the avatar/lip-sync layer never needs to know which provider is speaking.

```
Serein's response text
   ↓
Voice Router  (Azure primary / HeadTTS local fallback — user-configurable)
   ↓
TTS Engine → Audio stream + Timestamped viseme/phoneme data
   ↓
Viseme Mapper (phoneme/viseme IDs → ~10 grouped mouth-shape blend shapes)
   ↓
Playback loop: as audio.currentTime advances, apply matching blend-shape weight each frame
   ↓
Avatar mouth moves in sync with speech, driven by real timing data
```

Paired with an offline wake-word engine (e.g. openWakeWord) and local speech-to-text (Whisper/whisper.cpp), the full voice loop can run without any cloud dependency when the local tier is selected — matching the local-first principle end-to-end.

PART VII · SECTION 29

Emotion → Expression Mapping

```
Emotion Engine state  (happiness: 0.8, concern: 0.1, curiosity: 0.4, ...)
   ↓
Expression Mapper — maps dominant emotion(s) to the nearest VRM expression preset, blended not switched
   ↓
Blend shape weights applied each frame, layered underneath the active viseme (mouth) animation
   ↓
Avatar face reflects Serein's actual current emotional state in real time
```

Visemes (mouth, from speech) and expressions (brows/eyes, from emotion) are separate blend-shape groups on the VRM rig and run concurrently without conflict — she can look concerned while her mouth shapes track exactly what she's saying.

PART VIII · SECTION 30

Database Architecture & Full Schema

SQLite remains correct: single-user, local-first, lightweight, durable across restarts. This revision adds `scheduled_events` and extends `permissions` and `tasks` for the new control and scheduling layers.

Table	Purpose & key fields
identity	id, user_name, preferred_name, timezone, created_at, last_seen
personality_state	calm, mature, cheerful, energetic, protective, companion, professional
emotion_state	happiness, curiosity, care, concern, pride, affection, loneliness, sadness, jealousy, possessiveness
relationship_state	trust, familiarity, attachment, interaction_count, total_conversation_minutes
growth_state	technical_interest, study_interest, productivity_interest, gaming_interest, social_interest
memories	id, content, tags, importance, emotion, source, embedding, created_at
episodes	id, title, description, emotional_impact, created_at
goals	id, title, description, priority, status, created_at, completed_at
tasks	id, description, status, sandbox_path, requires_confirmation, started_at, completed_at, result_summary
scheduled_events new	id, trigger_type (absolute/relative/recurring/conditional), fires_at, recurrence_rule, condition_spec, message, status, created_at, fired_at
system_actions_log new	id, action_type, target, backend, initiated_by (voice/scheduled/proactive), permission_tier, result, timestamp — the audit trail for every Command Bus action
permissions extended	camera_access, microphone_access, file_access, internet_access, automation_access, **system_control_access (per Section 31 tiers)**, **proactive_speech_enabled**
system_events	event_type, description, timestamp

PART VIII · SECTION 31

Security, Privacy & Trust Model

Full system control raises the stakes considerably beyond a conversational assistant — this section is expanded specifically to match that.

Tiered permission model for system control

Tier	Covers	Confirmation required?
Read-only	Querying open apps, file listings, system status, clipboard read	No — always allowed once system_control_access is granted
Reversible action	Opening/closing apps, volume/ brightness changes, moving files within user directories	No, but always logged and always verbally confirmed after the fact
Destructive/ irreversible	Deleting files, killing processes with unsaved work, shutdown/restart, installing software	**Yes — explicit confirmation every time**, no "always allow" override
Sensitive domain	Camera, microphone, network calls to unfamiliar hosts, credentials	**Yes**, checked against the permissions table every single time, regardless of past approvals

Core rules

- API keys and secrets stay outside source code, excluded from version control.
- Memory is private user state; Serein never silently shares it.
- **At-rest encryption:** given the database accumulates years of personal, emotional, and now system-usage history, SQLite runs with encryption at rest (SQLCipher, a drop-in replacement).
- Every Command Bus action is written to `system_actions_log` (Section 30) — a full, reviewable audit trail of everything Serein has ever done to the real system, including what triggered it (voice, schedule, or proactive judgment).
- Self-modification of protected core systems — Identity, Memory, Relationship, Growth, Emotion, Safety, **and the Command Bus/Scheduler themselves** — requires explicit user approval; see the controlled patch flow below.
- Different users must have fully isolated state; no shared instance across users.

Controlled self-improvement

Level	Capability
0	Observe only
1	Suggest changes only
2	Generate/apply patches — only after user approval **recommended default**

3	Autonomous critical self-modification — **not allowed**

```
Detect issue or requested change → Analyze → Generate proposed patch
   ↓
Run tests → Show change/summary to user → User approval
   ↓
Apply → Record evolution log → Rollback available
```

PART VIII · SECTION 32

Recommended Final Build Order

#	Subsystem	Purpose / rationale
1	Core persistence	Identity, database integrity, memory foundation, including the new scheduled_events and system_actions_log tables from day one
2	AI brain layer + latency fix	Provider abstraction, router, conversation engine — proven sub-2s response before anything else is layered on
3	**Command Bus — read-only + reversible tiers**	new, moved early — application/window control and basic settings first, so system control is validated before it's trusted with anything destructive
4	**Scheduler — absolute/relative triggers**	new, moved early — get "remind me at 8pm" working end-to-end, including the Always-On Service split, before building anything more elaborate
5	Basic 3D avatar on screen	Static VRM model with simple expression switching — proves the "she's real" feeling early
6	Voice: TTS + amplitude lip sync	Basic spoken output, crude lip sync placeholder
7	Phoneme-based lip sync	Upgrade to the full Section 28 pipeline
8	Conversation continuity + memory retrieval	Short-term history, embedding-based relevant-memory search
9	Automatic learning	Background memory extraction, structured formation
10	Relationship + Emotion engines	Now wired to the avatar's expression system from steps 5–7
11	Growth engine	User-specific adaptation without altering core identity
12	**Command Bus — destructive/ sensitive tiers + Universal input fallback**	new — the higher-risk control tiers (Section 14, 15, 16) and confirmation flows, built only once the safe tiers are proven
13	**Proactive Intelligence**	new — generalizing the Scheduler into ambient noticing/speaking-up behavior, with guardrails from Section 18
14	Task queue + sandboxed coding-agent handoff	The build/run/test loop from Part V
15	Full shell/boot-replacement	Wayland compositor, full-screen boot-first Serein experience, Always-On Service formalized as a systemd service
16	Vision (optional)	Presence/activity/environmental awareness
17	Cross-device apps	A phone app talking to the same core service (Section 25) becomes straightforward once the split-process model exists
18	Web application	Deliberately last

19	Controlled self-evolution	User-approved patch/rollback system

PART VIII · SECTION 33

Complete Technology Stack Summary

Base OS
Debian or Arch Linux (minimal install)
No default desktop environment

Shell / compositor
Wayland + wlroots
Custom compositor owning the boot screen and window control

System control
D-Bus, systemd, PolicyKit, NetworkManager, AT-SPI
Application, process, hardware, and network control backends on the Command Bus

Universal input fallback
ydotool / wtype
Wayland-native synthetic input for apps with no API

Scheduler
Background timer/condition service (part of the Always-On core service)
Absolute, relative, recurring, and conditional triggers

Avatar creation
VRoid Studio → VRM export
Slider-based anime character design

Avatar runtime
Unity + UniVRM
Most mature VRM plugin ecosystem; Mixamo-retargeted idle animation

Voice output (primary)
Azure Neural TTS
Native phoneme-precise viseme output

Voice output (local/offline)
HeadTTS
Free, MIT-licensed, native phoneme timestamps + Oculus visemes

Speech-to-text
Whisper / whisper.cpp (local)
Pairs with an offline wake-word engine

LLM brain

Provider-agnostic router — cloud (NVIDIA/OpenRouter) + local (Ollama/llama.cpp)

Fast/smart/local model split

Coding-agent engine

Existing agentic coding-tool loop (reused, not reimplemented)

Runs inside the sandbox for build/run/test/fix

Sandbox

Container/VM per build task

Isolated scratch filesystem; confirmation required to cross the boundary

Memory retrieval

Embedding similarity search (sqlite-vss or in-process cosine search)

Replaces keyword matching

Persistence

SQLite + SQLCipher (encryption at rest)

Single-user, local-first, encrypted, includes full action audit log

PART VIII · SECTION 34

Testing Strategy & Definition of Done

Unit tests

- Database CRUD/schema integrity, including `scheduled_events` and `system_actions_log`.
- Identity, memory embedding/search/retrieval.
- Command Bus routing — each backend category dispatches correctly and independently.
- Scheduler trigger resolution (absolute/relative/recurring/conditional) and firing accuracy.
- Relationship/emotion/growth state transitions against their explicit update functions.

Integration tests

- User message → memory retrieval → LLM → response, end to end.
- Spoken system-control command → Command Bus → real action → confirmation, across each permission tier.
- **"Remind me at 8pm" set, avatar UI closed, core service restarted, reminder still fires at 8pm** — the specific regression test for the gap this revision closes.
- Build request → task queue → sandboxed agent → result → conversational report-back.
- Permission denial path for each control tier, confirming nothing silently escalates.

End-to-end tests

- Fresh installation, first boot, onboarding flow.
- A single compound command touching multiple backends at once ("close Chrome, mute audio, remind me in 20 minutes") executes all three correctly and confirms once.
- Offline/local model, local TTS, and local scheduler operation with no network present.
- A full "build me a tool for X" request from voice command to finished, tested, installed output.
- A proactive notice (e.g. low battery) fires correctly and respects the user's configured frequency/toggle settings.
- Persona-consistency eval: a fixed prompt set scored against a tone/warmth/refusal-style rubric, re-run after any model or provider swap.
- Self-update proposal → approval → patch → rollback, confirming the Command Bus and Scheduler remain outside auto-modification even after approval of unrelated patches.

Definition of a finished SereinOS

```
Power on → Always-On core service starts → Serein wakes → recognizes user → maintains continuity
   → understands current context → retrieves meaningful memories
   → reflects relationship/emotion/growth state on her face and in her voice
   → executes system commands, fires scheduled reminders, and notices things on her own
```

```
    → hands genuine build requests to the sandboxed agent
    → speaks and acts → learns from what mattered → persists it → grows over time
```

Serein is the product. SereinOS is her body. The AI model is her brain. Memory is her experience. Relationship is her bond. Growth is her evolution. Emotion is her dynamic state. The Command Bus and Agent system are her hands. The Scheduler is the part of her that doesn't forget, even in silence. Voice and the 3D avatar are her senses and her presence. The user is the reason she exists.

This document consolidates and supersedes SereinOS Master Specification Rev. 1 and Rev. 2, closing the proactive-behavior and full-system-control gaps identified in this revision session, current as of 9 August 2026.